

第三次理论作业

1. 试解答如下问题：
- (1) 构造一个不带回溯的自顶向下分析程序对文法有何要求？为什么？
- (2) 自顶向下分析不能处理左递归文法， 但为什么能处理其他递归文法？

(1) 要构造一个不带回溯的自顶向下分析程序，文法必须满足消除左递归和进行左因子化的要求。左递归会导致分析器陷入无限递归，无法继续分析输入，因而必须消除。左因子化则避免了多条产生式具有公共前缀时的二义性问题，使得分析器能够根据输入符号唯一地选择产生式。此外，文法应满足LL(1)条件，以确保在分析过程中每一步都能通过当前输入符号和预测分析表确定唯一的分析路径。

(2) 自顶向下分析无法处理左递归文法的原因在于，左递归会使分析器在展开产生式时陷入无限递归，导致无法继续分析输入。然而，对于右递归或非直接递归的文法，自顶向下分析是可以处理的。在右递归文法中，分析器会每次递归匹配并消费输入符号，递归深度与输入长度成正比，不会导致无限循环。对于间接递归文法，通过适当的改写或重写使其满足LL(1)条件，分析器也可以正确处理。这说明除了左递归外，自顶向下分析能够处理其他递归形式。

2. 有文法 G[A]:
- A ::= (B) | dB_e
- B ::= c | Bc
- 试设计自顶向下的语法分析程序。

因为要设计自顶向下的语法，要消除左递归。

```
B ::= c B'
B' ::= c B' | ε
```

```
procedure B;
begin
    if CLASS = 'c' then
        begin
            nextsym;
            BPrime;
        end;
    else
        error;
    end;

procedure BPrime;
begin
    if CLASS = 'c' then
        begin
            nextsym;
            Bprime;
        end
    else;
end;

procedure A;
```

```

begin
  if CLASS = '(' then
    begin
      nextsym;
      B;
      if CLASS = ')' then
        nextsym;
      else
        error;
      end;
    else if CLASS = 'd' then
      begin
        nextsym;
        B;
        if CLASS = 'e' then
          nextsym;
        else
          error;
        end;
      else
        error;
    end;
  end;
end;

```

3. 有文法 $G[Z]$:

$Z ::= AcB \mid Bd$

$A ::= AaB \mid c$

$B ::= aA \mid a$

- (1) 试求各选择（候选式）的 FIRST 集合；
- (2) 该文法的自顶向下的语法分析程序是否要编成递归子程序？为什么？
- (3) 试用递归下降分析法设计其语法分析程序。

(1)

FIRST(B)

$B ::= aA \mid a$

$\text{FIRST}(aA) = \{a\} \rightarrow \text{FIRST}(a) = \{a\} \rightarrow \text{FIRST}(B) = \{a\}$

FIRST(A)

$A ::= AaB \mid c$

$\text{FIRST}(c) = \{c\} \rightarrow \text{FIRST}(A) = \{c\}$

FIRST(Z)

$Z ::= AcB \mid Bd$

$\text{FIRST}(AcB) = \text{FIRST}(A) = \{c\}, \text{FIRST}(Bd) = \text{FIRST}(B) = \{a\}$

$\text{FIRST}(Z) = \text{FIRST}(AcB) \cup \text{FIRST}(Bd) = \{a, c\}$

(2)

该文法包含左递归，特别是在规则 $A ::= AaB$ 中， A 会递归调用自身。如果不消除左递归，使用自顶向下分析程序时会导致无限递归，最终使程序陷入死循环。因此，在实现自顶向下分析程序之前，必须先对文法进行左递归的消除，以确保分析过程能够顺利进行，不会出现无限递归的问题。

(3)

首先，消除非终结符 A 的左递归。原来的产生式 $A ::= AaB \mid c$

可以转换为：

```
A ::= c A'
A' ::= a B A' | ε
```

故

```
Z ::= A c B | B d
A ::= c A'
A' ::= a B A' | ε
B ::= a A | a
```

```
var
    currentToken: char;
    inputString: string;
    index: integer;

procedure nextToken();
begin
    if index <= length(inputString) then
    begin
        currentToken := inputString[index];
        index := index + 1;
    end
    else
        currentToken := #0;
    end;

procedure match(expected: char);
begin
    if currentToken = expected then
        nextToken()
    else
        writeln('Syntax Error: Expected ', expected, ', found ', currentToken);
    end;

procedure Z();
begin
    if currentToken = 'c' then
    begin
        A();
        match('c');
        B();
    end
```

```

    else if currentToken = 'a' then
    begin
        B();
        match('d');
    end
    else
        writeln('Syntax Error in Z: Unexpected token ', currentToken);
end;

procedure A();
begin
    if currentToken = 'c' then
    begin
        match('c');
        APrime();
    end
    else
        writeln('Syntax Error in A: Unexpected token ', currentToken);
end;

procedure APrime();
begin
    if currentToken = 'a' then
    begin
        match('a');
        B();
        APrime();
    end
    else;
end;

procedure B();
begin
    if currentToken = 'a' then
    begin
        match('a');
        A();
    end
    else
        writeln('Syntax Error in B: Unexpected token ', currentToken);
end;

```